



تحقیق شماره ۲

تکلیف پژوهشی: تحلیل جامع و پیاده‌سازی خط لوله تبدیل قالب‌های نشانه‌گذاری داده (Data Annotation Formats)

مقدمه:

در توسعه سیستم‌های بینایی ماشین، استراتژی ذخیره‌سازی و مدیریت برچسب‌ها (Labels) نقش حیاتی در کارایی و مقیاس‌پذیری آموزش مدل‌ها دارد. با گسترش کاربردهای هوش مصنوعی در حوزه‌هایی نظیر رباتیک، پزشکی و تحلیل تصاویر هوایی، قالب‌های داده از حالت‌های ساده فراتر رفته و نیازمند مدیریت متادیتاهای پیچیده‌تر شده‌اند.

صورت سوال:

این تکلیف از دو بخش تشکیل شده است: پژوهش و مقایسه تحلیلی قالب‌ها، و سپس توسعه یک ابزار نرم‌افزاری ماژولار برای تبدیل این قالب‌ها به یکدیگر.

بخش اول: پژوهش و بررسی انواع قالب‌های نشانه‌گذاری

دانشجویان موظف‌اند در پژوهش خود قالب‌های زیر را به تفصیل بررسی کرده و در قالب یک جدول جامع با یکدیگر مقایسه کنند:

الف) برخی قالب‌های تشخیص اشیاء (Object Detection) و سگمنتیشن:

- قالب YOLO (TXT format) (بررسی نسخه‌های مختلف مانند YOLOv5/v8)
- قالب COCO (JSON format)
- قالب PASCAL VOC (XML format)
- قالب DOTA (جهت پشتیبانی از جعبه‌های محصورکننده جهت‌دار - Oriented Bounding Boxes (OBB)، پرکاربرد در رباتیک و تصاویر هوایی)
- قالب Open Images (CSV) (مخصوص مجموعه داده‌های توزیع شده و مقیاس بزرگ)
- قالب TFRecord و HDF5 (برای مدیریت داده‌های پیچیده، چندوجهی (Multi-modal) و تصاویر پزشکی)
- قالب KITTI و Cityscapes (پرکاربرد در سیستم‌های ناوبری خودکار)

ب) برخی قالب‌های کلاس‌بندی تصویر (Image Classification):

- ساختار سلسله‌مراتبی پوشه‌ای (ImageNet / Folder-based style)
 - قالب نگاشت متنی/جدولی (CSV / JSON mapping)
 - فرمت‌های آرشیوی و بهینه‌شده برای خواندن سریع داده‌های عظیم ((LMDB و WebDataset (TAR))
- معیارهای مقایسه در جدول نهایی: ساختار ذخیره مختصات (مطلق/نسبی، HBB در برابر OBB)، تفاوت‌های بنیادین در سینتکس (JSON در برابر XML/TXT)، سربار حافظه (Storage Overhead)، سرعت بارگذاری در خط لوله آموزش

(I/O Bottleneck)، پشتیبانی از متادیتاهای اضافی (مانند زاویه، ماسک چندضلعی، و نقاط کلیدی)، و سازگاری بومی با فریم‌ورک‌های استاندارد (PyTorch, TensorFlow).

بخش دوم: توسعه ابزار تبدیل قالب مبتنی بر فایل پیکربندی (Config-driven Converter Pipeline)

تسک عملی:

یک اسکریپت پایتون استاندارد و شی‌گرا (OOP) توسعه دهید که وظیفه تبدیل قالب‌های مختلف نشانه‌گذاری به یکدیگر را بر عهده داشته باشد. این اسکریپت نباید Hardcode شود؛ بلکه باید تمام تنظیمات خود را از یک فایل پیکربندی (YAML یا JSON) بخواند.

الزامات پیاده‌سازی کد:

۱. طراحی فایل **Config**: فایل پیکربندی باید شامل مسیر پوشه تصاویر، مسیر فایل‌های برچسب ورودی، فرمت ورودی (مثلاً COCO)، فرمت خروجی (مثلاً YOLO)، مسیر ذخیره خروجی، و در صورت نیاز، نگاشت کلاس‌ها (Class Mapping) برای فیلتر کردن یا ادغام برخی کلاس‌ها باشد.

۲. مدل داده میانی (**Intermediate Representation**): کد شما باید ابتدا داده‌های ورودی را به یک ساختار استاندارد در حافظه (مانند یک کلاس Dataclass در پایتون شامل مختصات نرمالایز شده، $[cx, cy, w, h]$ و شناسه کلاس) تبدیل کند و سپس از این ساختار میانی، فایل خروجی را تولید نماید.

۳. مدیریت خطا (**Exception Handling**): هندل کردن مواردی مانند: تصاویری که فایل برچسب متناظر ندارند، مختصاتی که از ابعاد تصویر بیرون زده‌اند (Out of bounds)، و کلاس‌های نامعتبر یا تعریف‌نشده در فایل Config.

۴. کدنویسی تمیز: رعایت استانداردهای PEP8، استفاده از Type Hinting، و نوشتن Docstring برای کلاس‌ها و توابع الزامی است.

خروجی مورد انتظار این بخش: دانشجویان باید سورس کد (ترجیحاً در قالب یک مخزن یا فایل py. ساختاریافته) به همراه یک نمونه فایل config.yaml و یک اسکریپت تست کوچک ارائه دهند که نشان دهد به عنوان مثال، یک فایل حاوی ۱۰۰ برچسب COCO با موفقیت و مختصات صحیح به پوشه حاوی فایل‌های TXT با فرمت YOLO تبدیل شده است.